Week 7 Exercises

Exercise 1 `brm()`

Reproduce the familiar negative binomial regression below with `brm()`.

Hints:

1. See `?brms::brmsfamily` for the distributions available in `brm()`.
2. Use `#| results: hide` in your Quarto code chunks to hide the Stan junk output. It's pesky.

```
# load packages
library(glmmTMB)

# load data
holland <- crdata::holland2015

# formula corresponds to model 1 for each city in holland (2015) table 2
f <- operations ~ lower + vendors + budget + population

# fit poisson regression model for Santiago
fit <- glmmTMB(
  f,
  family = nbinom2,
  data = holland,
  subset = city == "santiago"
  )

# summary
summary(fit)
```

Exercise 2 Convergence

Using the output above, argue that `warmup` and `iter` are sufficiently large (or not).

Exercise 3 `brm()` arguments

Explain the following arguments to `brm()`. Explain what the argument controls and recommend reasonable defaults.

- `chains`
- `iter`
- `warmup`
- `cores`
- `backend`

Exercise 4 {rstan} or {cmdstanr}

Reproduce the fits from Exercise 1 using a `.stan` model via {rstan} or {cmdstanr}.

Hint: You might want to use `neg_binomial_2_log`–see the Stan docs.

Exercise 5 Experimenting with difficult posterior

The posterior below is somewhat difficult. The samples generate slowly and the effective sample size is relatively low.

Stan is a excellent with default settings, but there are ways to improve efficiency even further for difficult problems.

Here's *one way* to think about efficiency: you want your effective *samples per second* (of your time) to be relatively large. Experiment with the several strategies to increase this measure of efficiency.

1. Rescale the predictors to have mean of zero and SD of one or 0.5. Or perhaps have a range from zero to one.
2. Increase the number of chains run in parallel.
3. Increase the number of iterations (and the warmp).
4. Increase the `adapt_delta` parameter. Search `?brm` for "adapt_delta" for more.

```r
# load packages
library(brms)
library(posterior)

# load data
hks <- crdata::hks2013

# fit negative binomial model
f <- osvAll ~ troopLag + policeLag + militaryobserversLag +
  brv_AllLag + osvAllLagDum + incomp + epduration +
  lntpop
fit_mcmc <- brm(
  f,
  data = hks,
  family = negbinomial,
  warmup = 5000,
  iter = 10000,
  chains = 4,
  cores = 4)
```

```r
# print fit
fit_mcmc
```

```
 Family: negbinomial
  Links: mu = log
Formula: osvAll ~ troopLag + policeLag + militaryobserversLag + brv_AllLag + osvAllLagDum + incomp + epduration +
   Data: hks (Number of observations: 3746)
  Draws: 4 chains, each with iter = 10000; warmup = 5000; thin = 1;
         total post-warmup draws = 20000

Regression Coefficients:
                    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
Intercept              -9.24      0.90   -11.01    -7.48 1.00    15859
troopLag               -0.53      0.07    -0.65    -0.38 1.00    16305
policeLag              -9.92      1.03   -11.91    -7.87 1.00    19495
militaryobserversLag   21.88      1.39    19.32    24.78 1.00    17140
brv_AllLag              0.00      0.00    -0.00     0.00 1.00    22348
osvAllLagDum            2.18      0.18     1.84     2.54 1.00    19066
incomp                  2.37      0.18     2.02     2.73 1.00    17325
epduration             -0.00      0.00    -0.00     0.00 1.00    26620
lntpop                  0.71      0.08     0.55     0.86 1.00    16803
                    Tail_ESS
Intercept              15196
troopLag               13494
policeLag              13354
militaryobserversLag   13133
brv_AllLag             12765
```

```
osvAllLagDum            14991
incomp                  14748
epduration              15954
lntpop                  15186

Further Distributional Parameters:
      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
shape     0.06      0.00     0.05     0.06 1.00    23981    13413

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

How long it took.

We can grab the time (in seconds) it took to obtain the samples from the `fit` component of `fit_mcmc`.

```
# time (in seconds) it took to samples for each chain
times <- get_elapsed_time(fit_mcmc$fit)
times
```

```
        warmup sample
chain:1 24.911 16.318
chain:2 24.049 16.093
chain:3 25.260 14.937
chain:4 23.277 14.928
```

```
# find max of warmup + samples
t <- max(apply(times, 1, sum))
t
```

```
[1] 41.229
```

How many (effective) samples we generated.

We can compute the ESS using the `effective_samples()` function in {bayestestR} package. We need a way to think about the ESS overall, so I just averaged the bulk ESS across the parameters.

```
# load package; some helpful things here!
library(bayestestR)

# compute ess for each parameter
ess <- effective_sample(fit_mcmc)
ess
```

```
               Parameter   ESS ESS_tail
1            b_Intercept 15859    15196
2             b_troopLag 16305    13494
3            b_policeLag 19495    13354
4 b_militaryobserversLag 17140    13133
5           b_brv_AllLag 22348    12765
6         b_osvAllLagDum 19066    14991
7              b_incomp 17325    14748
8           b_epduration 26620    15954
9              b_lntpop 16803    15186
```

```
# compute the average ess for a single number
avg_ess <- mean(ess$ESS)
avg_ess
```

```
[1] 18995.67
```

Effective samples per second.

And then we can compute the ratio–effective samples per second.

```
avg_ess/t
```

```
[1] 460.7356
```

To make this easy, we can make a function to compute the effective samples per second.

```
sps <- function(fit) {
  times <- get_elapsed_time(fit$fit)
  t <- max(apply(times, 1, sum))
  ess <- effective_sample(fit)
  avg_ess <- mean(ess$ESS)
  list("sps" = avg_ess/t,
       "ess" = ess,
       "times" = times)
}
```